

4. If we wanted to have the left and right joins between the new_posts and category tables, we'd have to use the full outer join and write the following:

```

forumdb=> select c.pk,c.title,p.pk,p.title from categories c full
outer join new_posts p on p.category=c.pk;

```

pk	title	pk	title
1	Database	1	Indexing PostgreSQL
1	Database	2	Indexing Mysql
3	Programming Languages	3	Data types in C++
(NULL)	(NULL)	6	A new Book
2	Unix	(NULL)	(NULL)
5	Database	(NULL)	(NULL)
4	New Category	(NULL)	(NULL)

(7 rows)

This diagram illustrates how the full outer join works:

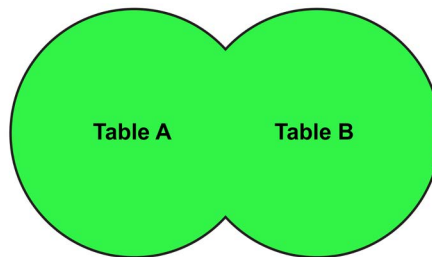


Figure 5.5: A full outer join

One question we need to consider is, *What is the difference between a full join and a cross join, which we saw at the beginning of this section on joins?*

Well, a full outer join is different from a cross join because a cross join makes a Cartesian product from all the records present in the tables.

For example, in a cross join with the same data as the preceding full join, we would get the following result:

```

forumdb=> select c.pk,c.title,p.pk,p.title from categories c cross join
new_posts p;

```

pk	title	pk	title
----	-------	----	-------

```

1 | Database          | 1 | Indexing PostgreSQL
2 | Unix              | 1 | Indexing PostgreSQL
3 | Programming Languages | 1 | Indexing PostgreSQL
4 | New Category      | 1 | Indexing PostgreSQL
5 | Database          | 1 | Indexing PostgreSQL
1 | Database          | 2 | Indexing Mysql
2 | Unix              | 2 | Indexing Mysql
3 | Programming Languages | 2 | Indexing Mysql
4 | New Category      | 2 | Indexing Mysql
5 | Database          | 2 | Indexing Mysql
1 | Database          | 3 | Data types in C++
2 | Unix              | 3 | Data types in C++
3 | Programming Languages | 3 | Data types in C++
4 | New Category      | 3 | Data types in C++
5 | Database          | 3 | Data types in C++
1 | Database          | 6 | A new Book
2 | Unix              | 6 | A new Book
3 | Programming Languages | 6 | A new Book
4 | New Category      | 6 | A new Book
5 | Database          | 6 | A new Book
(20 rows)

```

Using LATERAL JOIN

A lateral join is a type of join in SQL that allows you to join a table with a subquery, where the subquery is run for each row of the main table. The subquery is executed before joining the rows and the result is used to join the rows. With this join mode, you can use information from one table to filter or process data from another table.

Let's add a field called likes to the table posts and insert some data on this field:

```

forumdb=> alter table posts add likes integer default 0;
ALTER TABLE
forumdb=> update posts set likes = 3 where title like 'Indexing%';
UPDATE 2

```

The current situation is:

```

forumdb=> select title,likes from posts order by likes ;
      title          | likes

```

```
-----+-----
Data types in C++ |      0
Indexing PostgreSQL |    3
Indexing Mysql    |    3
(3 rows)
```

Now let's suppose that we want to search for all users that have posts with likes greater than 2; a query that solves this problem is:

```
forumbd=> select u.* from users u where exists (select 1 from posts p
where u.pk=p.author and likes > 2 ) ;
pk | username | gecos | email
----+-----+-----+-----
1 | luca_ferrari |      | luca@pgtraining.com
(1 row)
```

Let's suppose now that we want the value of the likes field too. A simple way to solve this problem is using the lateral join:

```
forumbd=> select u.username,q.* from users u join lateral (select author,
title,likes from posts p where u.pk=p.author and likes > 2 ) as q on true;
username | author | title | likes
-----+-----+-----+-----
luca_ferrari | 1 | Indexing PostgreSQL | 3
luca_ferrari | 1 | Indexing Mysql | 3
(2 rows)
```

This query is very similar to the EXISTS query, except the fact that, in the main query, we can have all the values that are in the subquery and we can use them in the main part of the query.

Aggregate functions

Aggregate functions perform a calculation on a set of rows and return a single row. PostgreSQL provides all the standard SQL aggregate functions:

- **AVG()**: This function returns the average value.
- **COUNT()**: This function returns the number of values.
- **MAX()**: This function returns the maximum value.
- **MIN()**: This function returns the minimum value.
- **SUM()**: This function returns the sum of values.

Aggregate functions are used in conjunction with the `group by` clause. A `group by` clause splits a resultset into groups of rows and aggregate functions perform calculations on them. For example, if we wanted to count how many records there are for each category, PostgreSQL first groups the data and then counts it. The following diagram illustrates the process:

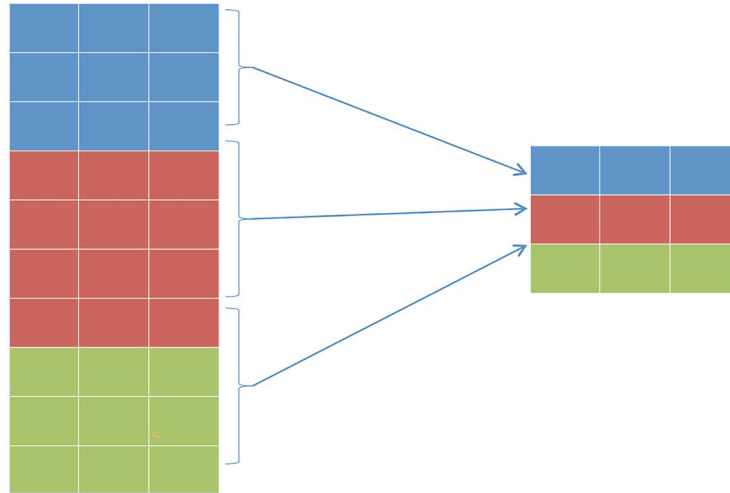


Figure 5.6: Group by aggregation

This diagram illustrates that PostgreSQL, before grouping the data, sorts it internally. Therefore, we must remember that a grouping operation always implies an ordering operation; this will become more clear when we discuss performance later on.

Now that we have understood the theory, let's address how to actually calculate how many records there are for each category:

```
forumdb=> select category,count(*) from posts group by category;
category | count
-----+-----
3        | 1
1        | 2
(2 rows)
```

The preceding query counts how many records there are for each category in the `posts` table.

Another way to write the same query is as follows:

```
forumdb=> select category,count(*) from posts group by 1;
category | count
```

```

-----+-----
      3 |      1
      1 |      2
(2 rows)

```



In PostgreSQL, we can write the `group by` condition using the name of the fields or their position in the query.

Another condition that we can use is the `having` condition. Suppose that we want to count how many records there are for each category that have a count greater than 2. To do this, we would have to add the `having` condition after the `group by` condition, thus writing the following:

```

forumdb=> select category,count(*) from posts group by category having
count(*) > 1;
category | count
-----+-----
      1 |      2
(1 row)

```

Similarly, we could do this:

```

forumdb=> select category,count(*) from posts group by 1 having count(*) >
1;
category | count
-----+-----
      1 |      2
(1 row)

```

Now let's see how the aggregation functions work if we add aliases. Let's resume the first query and write the following:

```

forumdb=> select category,count(*) as category_count from posts group by
category;
category | category_count
-----+-----
      3 |              1
      1 |              2
(2 rows)

```

As seen here, we can use an alias on aggregate functions.

However, what do we do if we want to use an alias inside a query that has a having condition too? To answer this question, let's try the following statement:

```
forumdb=> select category,count(*) as category_count from posts group by
category having category_count > 1;
ERROR: column "category_count" does not exist
```

As we can see, we can't use an alias on a having condition. The correct way to write the preceding query is as follows:

```
forumdb=> select category,count(*) as category_count from posts group by
category having count(*) > 1;
category | category_count
-----+-----
1 | 2
(1 row)
```

In the next chapter, we will discuss aggregates in more detail.

UNION/UNION ALL

The UNION operator is used to combine the resultset of two or more SELECT statements. We can use the UNION statement only if the following rules are respected:

- Each SELECT statement within UNION must have the same number of columns.
- The columns must have similar data types.
- The columns in each SELECT statement must be in the same order.

Let's explore an example.

First, we need to insert some data:

```
forumdb=> insert into tags (tag) values ('Database'),('Operating
Systems');
INSERT 0 2
```

The situation on the table tags is:

```
forumdb=> select tag from tags;
tag
-----
```

```
Database
Operating Systems
(2 rows)
```

and on the table categories is:

```
forumdb=> select title from categories;
          title
-----
Database
Unix
Programming Languages
New Category
Database
(5 rows)
```

Suppose now that we want to have a resultset that is a union of tags and categories; in other words, we want to reach this result:

```
Operating Systems
Database
New Category
Programming Languages
Unix
```

To achieve this, we have to use the UNION operator:

```
forumdb=> select tag as datalist from tags UNION select title as datalist
from categories;
          datalist
-----
New Category
Operating Systems
Programming Languages
Database
Unix
(5 rows)
```

The UNION operator combines the values of the two tables and removes duplicates. If we don't want duplicates to be removed and instead have them remain in the resultset, we have to use the UNION ALL operator:

```
forumdb=> select tag as datalist from tags UNION ALL select title as
          datalist from categories order by 1;
          datalist
-----
Database
Database
Database
New Category
Operating Systems
Programming Languages
Unix
(7 rows)
```

EXCEPT/INTERSECT

The EXCEPT operator returns rows by comparing the resultsets of two or more queries. The EXCEPT operator returns distinct rows from the first (left) query that is not in the output of the second (right) query. Similar to the UNION operator, the EXCEPT operator can also compare queries that have the same number and the same datatype of fields.

For example, say we have the following:

```
forumdb=> select tag from tags;
          tag
-----
Database
Operating Systems
(2 rows)

forumdb=> select title from categories;
          title
-----
Database
```

```

Unix
Programming Languages
New Category
Database
(5 rows)

```

And we want to reach this result:

```

New Category
Programming Languages
Unix

```

We would need to order all records that are present in the categories table but that are not present in the tags table by the title field. To do this, we would use the following query:

```

forumdb=> select title as datalist from categories except select tag as
          datalist from tags order by 1;
          datalist
-----
New Category
Programming Languages
Unix
(3 rows)

```

The INTERSECT operator performs the reverse operation. It searches for all the records present in the first table that are also present in the second table:

```

forumdb=> select title as datalist from categories intersect select tag as
          datalist from tags order by 1;
          datalist
-----
Database
(1 row)

```

In this section, we have taken a detailed look at the instructions needed to search data in tables using various statements and joins. In the next section, we will see how to modify the data in the tables in more advanced ways.

Using UPSERT

In this section, we will look at the PostgreSQL way to make an UPSERT statement. There is no UPSERT statement in SQL, but the same effect can be achieved using an INSERT SQL statement.

UPSERT – the PostgreSQL way

In PostgreSQL, the UPSERT statement does not exist as in other DBMSes. An UPSERT statement is used when we want to insert a new record on top of the existing record or update an existing record. To do this in PostgreSQL, we can use the ON CONFLICT keyword:

```
INSERT INTO table_name(column_list) VALUES(value_list)
ON CONFLICT target action;
```

Here, ON CONFLICT means that the target action is executed when the record already exists (meaning when a record with the same primary key exists). The target action could be this:

```
DO NOTHING
```

Alternatively, it could be the following:

```
DO UPDATE SET { column_name = { expression | DEFAULT } |
    ( column_name [, ...] ) = [ ROW ] ( { expression | DEFAULT } [, ...]
) |
    ( column_name [, ...] ) = ( sub-SELECT )
} [, ...]
[ WHERE condition ]
```

Now, let's look at an example to better understand how UPSERT works:

1. For example, start with the j_posts_tags table:

```
forumdb=> \d j_posts_tags
               Table "forum.j_posts_tags"
  Column   | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 tag_pk    | integer |           | not null |
 post_pk   | integer |           | not null |
Foreign-key constraints:
    "j_posts_tags_post_pk_fkey" FOREIGN KEY (post_pk) REFERENCES
posts(pk)
```